

Reinforcement Learning

Temporal-Difference Learning

Sutton/Barto* Chapter 6

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction,
2nd edition, MIT Press, Cambridge, MA, 2018



Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a
	expected immediate reward from state s after action a
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
Q, Q_t	array estimates of action-value function q_π or q_*

Temporal Differencing

U_t	target for estimate at time t
δ_t	TD error

Temporal-Difference (TD) Learning

- We will first focus on:
 - On-step
 - Tabular
 - Model-free TD
- **Idea:** incrementally make the state value estimate $V(S_t)$ a little more similar to the immediate reward and the discounted value of the next state
$$R_{t+1} + \gamma V(S_{t+1})$$
- The difference is called the **DT error**:
$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$
- Called temporal difference because it uses differences between successive predictions made over time at t and $t + 1$.
- Combines ideas from MC and DP
 - From MC: learn from raw experience
 - From DP: calculates the error between two value estimates without waiting for the end of the episode.

TD Prediction

Estimate the value function for a policy.

TD Prediction

- Follow π and update estimate V of v_π

MC Error over complete episode

- MC: $V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$

Compare with the
incremental
Implementation of
MC methods.

TD Error for one step

- TD: $V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

Uses reward + estimate for next state = target U_t

- Updating an estimate using another estimate is called
“**bootstrapping.**”

Alg. Tabular TD(0)

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

Note: We can only update for time t at time $t + 1$ since we need S' and the reward R_{t+1} !

Advantages of TD Prediction Methods

- **Online updates** at each step. Bootstraps using estimate $V(S')$ for the value of the next state.
MC has to wait for the end of the episode.
- **Model-free**. Only needs samples.
DP needs the transition probabilities!
- **Convergence**: For any fixed π , TD(0) converges to v_π with probability 1 if α decreases sufficiently slowly over time.
Converges typically faster than MC prediction.

Sarsa: On-policy TD Control

Find a good policy.

Sarsa

- **Idea:** Use GPI with TD methods for policy evaluation (prediction).
- **Evaluation:** Estimate $q_\pi(s, a)$ using the Q -table for current behavior π like TD prediction for V .

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

$Q(S_{t+1}, A_{t+1})$ for a terminal state is defined as 0.

- **Prediction:** Create a soft policy based on Q .
- **Note:** Update uses $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ which leads to the name **Sarsa**.

Alg. Sarsa

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Q-learning: Off-policy TD Control

Find a good policy.

Q-Learning

- An extremely influential method developed by Watkins (1989).

- Update rule only uses Sars:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The next action has not yet been observed. It is greedily chosen!

- Q directly approximates q_* independent of the behavior policy being followed! → Off-policy
- Converges to the optimal value function:
 - State-action pairs are represented discretely (in a table)
 - All action are repeatedly sampled in all states (keeps exploring).

Alg. Q-Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Q-Learning's Maximization Bias

- Q-learning update: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
- The next action is not observed, but Q -values are updated by choosing the next action as the maximum over Q -values. This cycle creates a **maximization bias** that leads to overestimating the Q -values.
- Solution: **double Q-learning** by learning two independent Q -functions and using one for the action selection and the other one for the bootstrap estimation.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, such that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using the policy ε -greedy in $Q_1 + Q_2$

 Take action A , observe R, S'

 With 0.5 probability:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha \left(R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A) \right)$$

 else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha \left(R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A) \right)$$

$S \leftarrow S'$

 until S is terminal

Bootstrap
estimation

Action
selection

Expected Sarsa

Expected Sarsa

- Like Q-learning, but use the expectation over all actions instead of $\max_a$

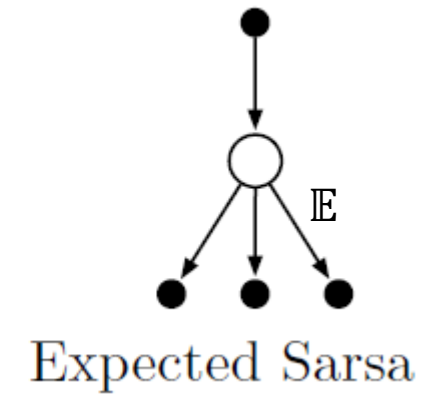
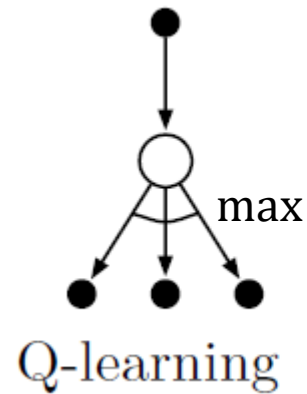
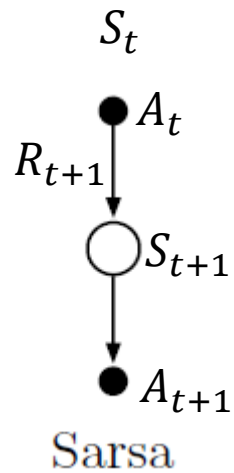
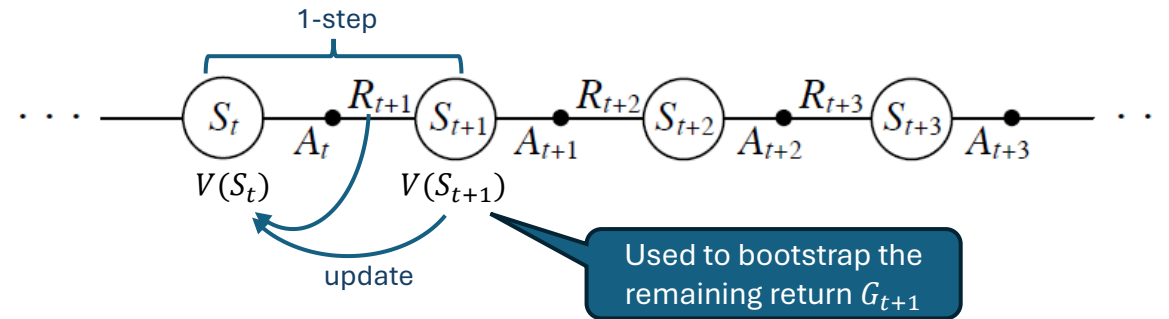
$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \mathbb{E}_\pi [Q(S_{t+1}, A_{t+1}) | S_{t+1}] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right] \end{aligned}$$

- **Properties:**

- Moves in the direction of Sarsa (in expectation).
- Eliminates Sarsa's variance due to the random selection of A_{t+1} .
- Can set the learning rate $\alpha = 1$ since it uses expectation!
- Can be used off-policy $\rightarrow$ generalizes Q-learning.

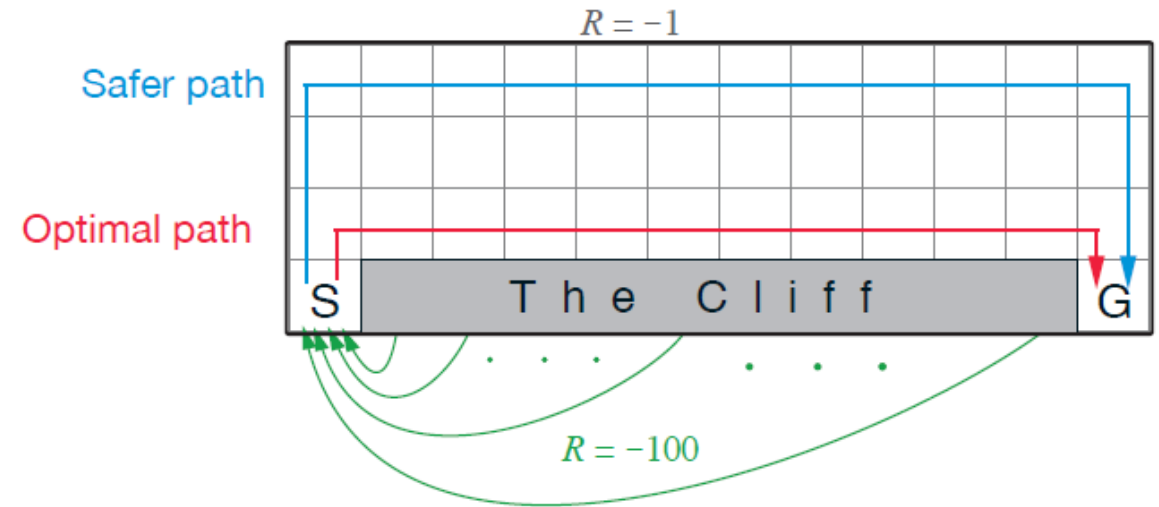
Comparison

Comparison: 1-step Backup Diagrams

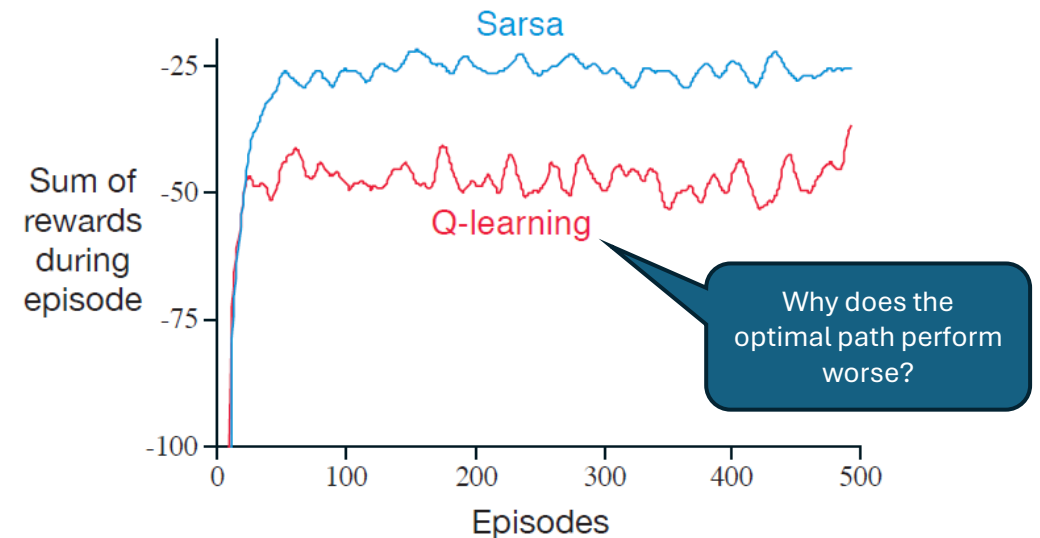


Sarsa vs. Q-Learning

- **Cliff Walking:** Consider the gridworld shown to the right.
- This is a standard undiscounted, episodic task, with start and goal states.
- Actions causing movement up, down, right, and left.
- Reward is -1 on all transitions except those into the region marked “The Cliff.” Stepping into this region incurs a reward of -100 and sends the agent instantly back to the start.



Performance of the Sarsa and Q-learning methods with ϵ -greedy action selection, $\epsilon = 0.1$.



Experimental Comparison

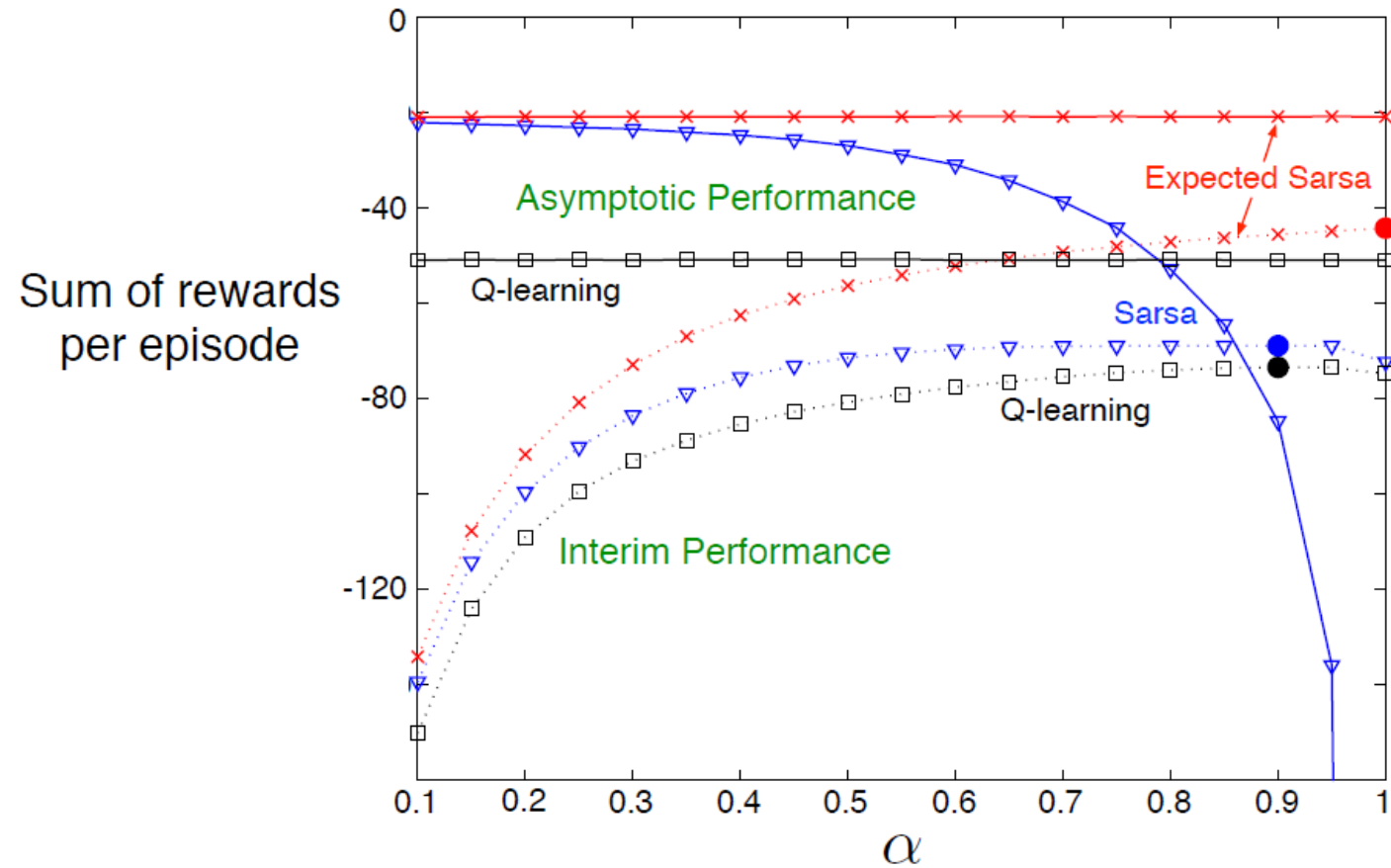


Figure 6.3: Interim and asymptotic performance of TD control methods on the cliff-walking task as a function of α . All algorithms used an ε -greedy policy with $\varepsilon = 0.1$. Asymptotic performance is an average over 100,000 episodes whereas interim performance is an average over the first 100 episodes. These data are averages of over 50,000 and 10 runs for the interim and asymptotic cases respectively. The solid circles mark the best interim performance of each method. Adapted from van Seijen et al. (2009).

Summary

- Time differencing (TD) is an alternative to MC for the prediction (policy evaluation) problem.
- **Method:** Stepwise reduces the TD error for the next step.
- **Issue:** Uses experience so it needs to maintain exploration
 - On-policy: Sarsa
 - Off-policy: Q-learning
- Q-learning and Sarsa are the most used RL methods.
 - Learn online (after each observation)
 - Simple with minimal computation
- We considered one-step, tabular, model-free TD. We will extend the TD ideas to ***n*-steps** and **function approximation** instead of tables.